

ModuleAutoDialerManage Developer Guide

Bit Dream IT

- [ModuleAutoDialerManage — Developer Guide](#)

- [Table of Contents](#)
- [1. Authentication](#)
 - [For external \(off-PBX\) clients](#)
 - [For local \(on-PBX\) clients](#)
- [2. Response envelope](#)
- [3. Campaigns \(tasks\) API](#)
 - [List all campaigns](#)
 - [Get a single campaign](#)
 - [Create a campaign](#)
 - [Update a campaign](#)
 - [Delete a campaign](#)
 - [Signal close \(batch operation\)](#)
- [4. Results API](#)
 - [Get call results \(incremental\)](#)
 - [Get poll results \(incremental\)](#)
- [5. Surveys \(polling\) API](#)
 - [List all surveys](#)
 - [Get a single survey](#)
 - [Create a survey](#)
 - [Delete a survey](#)
- [6. Audio files API](#)
 - [Upload an audio file](#)
 - [List audio files](#)
 - [Delete an audio file](#)
- [7. DNC blacklist API](#)
 - [Add number\(s\) to blacklist](#)
 - [List blacklist \(paginated\)](#)
 - [Delete from blacklist](#)
- [8. Dashboard / live status API](#)
 - [Campaign live status](#)
 - [Agent status panel](#)

- Campaign summary report
- Test call (single number preview)
- 9. CSV import/export API
 - Export campaign results as CSV
 - Import numbers from CSV
- 10. Recordings API
 - Get recording file path for a call
- 11. Webhooks
 - Event 1: `campaign.state_changed`
 - Event 2: `campaign.completed`
 - Delivery guarantees
 - Sample Laravel receiver (handles both events)
 - Reconciling missed events
- 12. Laravel integration cookbook
 - Setup
 - Service class
 - Usage examples
- 13. Plain PHP integration
- 14. Python integration
- 15. Error handling
 - Common errors
 - Defensive client pattern
- 16. Rate limiting & performance
 - Recommended polling intervals
 - Concurrency limits
 - Database performance
- 17. Schema reference
 - Tasks table
 - Blacklist table
 - TaskResults table
 - PolingResults table

- [API endpoint summary](#)

ModuleAutoDialerManage — Developer Guide

REST API reference + integration examples **Module version:** 1.35 (Bit Dream IT edition) **Base URL:** `http://{pbx-host}/pbxcore/api/module-dialer-manage/v1` **License:** GPL-3.0-or-later

This module exposes a complete REST API for managing campaigns, surveys, results, and the DNC blacklist. Any HTTP client (Laravel `Http::`, Guzzle, raw `curl`, Python `requests`) works without an SDK.

Table of Contents

1. [Authentication](#)
 2. [Response envelope](#)
 3. [Campaigns \(tasks\) API](#)
 4. [Results API](#)
 5. [Surveys \(polling\) API](#)
 6. [Audio files API](#)
 7. [DNC blacklist API](#)
 8. [Dashboard / live status API](#)
 9. [CSV import/export API](#)
 10. [Recordings API](#)
 11. [Webhooks](#)
 12. [Laravel integration cookbook](#)
 13. [Plain PHP integration](#)
 14. [Python integration](#)
 15. [Error handling](#)
 16. [Rate limiting & performance](#)
 17. [Schema reference](#)
-

1. Authentication

For external (off-PBX) clients

Start a session, then reuse the cookie:

```
# 1. Login
curl -c cookies.txt -X POST http://PBX/admin-cabinet/session/start \
  -d "login=admin&password=YOUR_PASSWORD"

# 2. Subsequent requests reuse the cookie
curl -b cookies.txt http://PBX/pbxcore/api/module-dialer-manage/v1/task
```

For local (on-PBX) clients

Requests from `127.0.0.1` skip authentication entirely. Perfect for a Laravel app running on the same MikoPBX box.

```
// In a Laravel controller, no auth needed:
$response = Http::get('http://127.0.0.1/pbxcore/api/module-dialer-manage/v1/task');
```

2. Response envelope

All responses (except CSV export which returns a file) use this shape:

```
{
  "result": true,
  "data": { ... },
  "messages": []
}
```

Or on error:

```
{
  "result": false,
  "data": null,
  "messages": ["Task 42 not found"]
}
```

Always check the `result` field, not the HTTP status code (which is always 200 for JSON responses).

3. Campaigns (tasks) API

List all campaigns

```
GET /pbxcore/api/module-dialer-manage/v1/task
```

Response:

```
{
  "result": true,
  "data": {
    "results": [
      {
        "id": 42,
        "crmId": "1001",
        "name": "Payment reminders - June",
        "state": "0",
        "innerNum": "200",
        "innerNumType": "exten",
        "maxCountChannels": 5,
        "dialPrefix": "999",
        "timeStart": 540,
        "timeEnd": 1080,
        "scheduleDays": "1,2,3,4,5",
        "amdEnabled": 1,
        "callbackUrl": "https://crm.example.com/webhooks/dialer",
        "maxAttempt": 3,
        "tryInterval": 300
      }
    ]
  }
}
```

Get a single campaign

```
GET /pbxcore/api/module-dialer-manage/v1/task/{id}
```

Create a campaign

```
POST /pbxcore/api/module-dialer-manage/v1/task
Content-Type: application/json
```

```
{
  "crmId": "1001",
  "name": "Payment reminders - June",
  "description": "Outstanding June invoices",
  "state": 0,
  "innerNum": "200",
  "innerNumType": "exten",
  "maxCountChannels": 5,
  "dialPrefix": "999",
  "timeStart": 540,
  "timeEnd": 1080,
  "scheduleDays": "1,2,3,4,5",
  "maxAttempt": 3,
  "tryInterval": 300,
  "amdEnabled": 1,
  "callbackUrl": "https://crm.example.com/webhooks/dialer",
  "numbers": [
    {"number": "7912345678", "params": {"speech": "Your balance is 1000 rubles."}},
    {"number": "7912345679", "params": {"speech": "Your balance is 500 rubles."}}
  ]
}
```

Field reference:

Field	Type	Required	Default	Description
crmId	string	yes	-	External CRM ID
name	string	yes	-	Campaign name
description	string	no	""	Optional description
state	int	yes	0	0=open, 1=closed, 2=paused
innerNum	string	yes	-	Extension to bridge to
innerNumType	string	yes	"exten"	exten or polling
maxCountChannels	int	yes	1	Max simultaneous calls
dialPrefix	string	no	""	Outbound prefix
timeStart	int	no	0	Start time (minutes from midnight)
timeEnd	int	no	1440	End time (minutes from midnight)
scheduleDays	string	no	""	ISO weekdays (1-7, comma-separated)
maxAttempt	int	no	1	Retry attempts
tryInterval	int	no	60	Seconds between retries
amdEnabled	int	no	0	1 = enable AMD
callbackUrl	string	no	""	Webhook URL on completion
numbers	array	yes	-	Array of {number, params}

Response:

```
{
  "result": true,
  "data": {
    "id": 42,
    "taskId": 42
  }
}
```

Update a campaign

```
PUT /pbxcore/api/module-dialer-manage/v1/task/{id}
Content-Type: application/json

{"state": 2}
```

Any subset of fields is accepted. Common updates: - Pause: `{"state": 2}` - Resume: `{"state": 0}` - Close: `{"state": 1}` - Increase concurrency: `{"maxCountChannels": 10}` - Enable AMD: `{"amdEnabled": 1}` - Set webhook: `{"callbackUrl": "https://..."}`

Delete a campaign

```
DELETE /pbxcore/api/module-dialer-manage/v1/task/{id}
```

Deletes the campaign AND all its call results. Numbers in `m_Clients` are NOT deleted.

Signal close (batch operation)

```
POST /pbxcore/api/module-dialer-manage/v1/task-signal-close
Content-Type: application/json

{"taskIds": [42, 43, 44]}
```

Closes multiple campaigns in one call.

4. Results API

Get call results (incremental)

```
GET /pbxcore/api/module-dialer-manage/v1/results/{changeTime}
```

- `changeTime` is a Unix timestamp
- Returns only results newer than `changeTime`

- After processing, advance cursor to the latest result's `changeTime`

Response:

```
{
  "result": true,
  "data": {
    "results": [
      {
        "id": 1234,
        "task_id": 42,
        "number": "7912345678",
        "outDialState": "ANSWER",
        "inDialState": "ANSWER",
        "duration": 47,
        "attempt": 1,
        "time": "2025-07-25 15:30:00",
        "changeTime": 1721904600,
        "cause": "Normal Clearing",
        "linkedId": "PBX-1234567.890"
      }
    ]
  }
}
```

`outDialState` values:

State	Meaning
DIAL	Currently dialing
ANSWER	Call answered (successful)
NOANSWER	No answer
BUSY	Line busy
CHANUNAVAIL	Channel unavailable
CONGESTION	Network congestion
CANCEL	Call canceled
DONTCALL	Number blacklisted (DNC)

Get poll results (incremental)

```
GET /pbxcore/api/module-dialer-manage/v1/polling-results/{changeTime}
```

Same pattern as `/results` . Returns:

```
{
  "result": true,
  "data": {
    "results": [
      {
        "id": 5678,
        "pollingId": 1,
        "questionId": 1,
        "crmId": 100,
        "number": "7912345678",
        "value": "excellent",
        "key": "1",
        "time": "2025-07-25 15:31:00",
        "changeTime": 1721904660
      }
    ]
  }
}
```

5. Surveys (polling) API

List all surveys

```
GET /pbxcore/api/module-dialer-manage/v1/polling
```

Get a single survey

```
GET /pbxcore/api/module-dialer-manage/v1/polling/{id}
```

Create a survey

```
POST /pbxcore/api/module-dialer-manage/v1/polling
Content-Type: application/json

{
  "name": "Customer satisfaction",
  "questions": [
    {
      "id": 1,
      "text": "Press 1 for yes, 2 for no.",
      "lang": "en",
      "actions": [
        {"key": "1", "value": "yes", "nextQuestion": 2},
        {"key": "2", "value": "no", "nextQuestion": 2}
      ]
    },
    {
      "id": 2,
      "text": "Thank you for your feedback.",
      "lang": "en"
    }
  ]
}
```

Delete a survey

```
DELETE /pbxcore/api/module-dialer-manage/v1/polling/{id}
```

6. Audio files API

Upload an audio file

```
POST /pbxcore/api/module-dialer-manage/v1/audio
Content-Type: multipart/form-data

file: < WAV or MP3 file >
```

List audio files

```
GET /pbxcore/api/module-dialer-manage/v1/audio
```

Delete an audio file

```
DELETE /pbxcore/api/module-dialer-manage/v1/audio/{name}
```

7. DNC blacklist API

Add number(s) to blacklist

```
POST /pbxcore/api/module-dialer-manage/v1/blacklist
Content-Type: application/json

{
  "numbers": ["7912345678", "7912345679"],
  "reason": "Customer complaint",
  "source": "complaint"
}
```

Or single:

```
{"number": "7912345678", "reason": "Manual block", "source": "manual"}
```

Sources: `manual`, `complaint`, `regulator`, `auto-amd`

Response:

```
{
  "result": true,
  "data": {
    "added": 2,
    "skipped_duplicates": 0
  }
}
```

List blacklist (paginated)

```
GET /pbxcore/api/module-dialer-manage/v1/blacklist?limit=100&offset=0&q=7912
```

Query parameters: - `limit` (int, default 100, max 1000) — page size - `offset` (int, default 0) — pagination offset - `q` (string, optional) — partial number search

Response:

```
{
  "result": true,
  "data": {
    "entries": [
      {
        "id": 1,
        "number": "7912345678",
        "reason": "Customer complaint",
        "source": "complaint",
        "createdAt": 1721904600
      }
    ],
    "total": 1,
    "limit": 100,
    "offset": 0
  }
}
```

Delete from blacklist

```
DELETE /pbxcore/api/module-dialer-manage/v1/blacklist/{number}
```

Response:

```
{
  "result": true,
  "data": {"deleted": "7912345678"}
}
```

8. Dashboard / live status API

These endpoints are designed for 3-5 second polling from a dashboard UI.

Campaign live status

```
GET /pbxcore/api/module-dialer-manage/v1/task/{id}/status
```

Response:

```
{
  "result": true,
  "data": {
    "task_id": 42,
    "name": "Payment reminders - June",
    "state": 0,
    "state_label": "open",
    "in_progress": 3,
    "max_channels": 5,
    "total_dialed": 872,
    "amd_enabled": 1,
    "updated_at": "2025-07-25T15:30:00+03:00"
  }
}
```

Agent status panel

```
GET /pbxcore/api/module-dialer-manage/v1/agents-status
```

Response:

```
{
  "result": true,
  "data": {
    "agents": [
      {
        "number": "200",
        "name": "Agent Smith",
        "state": "Idle",
        "state_label": "idle",
        "is_idle": true
      },
      {
        "number": "201",
        "name": "Agent Jones",
        "state": "Up",
        "state_label": "in_call",
        "is_idle": false
      }
    ],
    "count": 2
  }
}
```

Campaign summary report

```
GET /pbxcore/api/module-dialer-manage/v1/task/{id}/summary
```

Response:

```
{
  "result": true,
  "data": {
    "task_id": 42,
    "name": "Payment reminders - June",
    "total_dialed": 1500,
    "answered": 872,
    "failed": 628,
    "answer_rate": 58.13,
    "avg_duration_sec": 47,
    "total_duration_sec": 40984,
    "state": 1,
    "state_label": "closed",
    "amd_enabled": 1
  }
}
```

Test call (single number preview)

```
POST /pbxcore/api/module-dialer-manage/v1/task/{id}/test-call
Content-Type: application/json
```

```
{
  "number": "7912345678",
  "extension": "200",
  "params": {"speech": "This is a test call."}
}
```

Creates a temporary one-call campaign with the same settings as task `{id}` and dials the specified number. Useful for previewing a poll before launching the full campaign.

9. CSV import/export API

Export campaign results as CSV

```
GET /pbxcore/api/module-dialer-manage/v1/task/{id}/export
```

Returns a CSV file with columns: `number`, `state`, `duration_sec`, `attempt`, `time`, `cause`. UTF-8 with BOM (auto-detected by Excel).

Response headers:

```
Content-Type: text/csv; charset=utf-8
Content-Disposition: attachment; filename="task_42_results.csv"
```

Import numbers from CSV

```
POST /pbxcore/api/module-dialer-manage/v1/task/{id}/import-csv
Content-Type: multipart/form-data

file: < CSV file >
```

CSV format (header row required):

```
number,name,params
7912345678,Ivan Ivanov,"{"speech":"Your balance is 1000"}"
7912345679,Petr Petrov,"{"speech":"Your balance is 500"}"
```

- **number** (required) — phone number
- **name** (optional) — caller name
- **params** (optional) — JSON string OR plain text (becomes `{text: "..."}`)

Response:

```
{
  "result": true,
  "data": {
    "task_id": 42,
    "rows_read": 2,
    "rows_added": 2,
    "detail": {...}
  }
}
```

10. Recordings API

Get recording file path for a call

```
GET /pbxcore/api/module-dialer-manage/v1/recording/{linkedId}
```

Joins to MikoPBX's core `CallDetailRecords` table by `linkedId` .

Response:


```
{
  "result": true,
  "data": {
    "linked_id": "PBX-1234567.890",
    "recording_path": "/storage/usbdisk1/mikopbx/astspool/monitor/2025/07/25/...",
    "recording_exists": true,
    "duration": 47,
    "dialstatus": "ANSWERED",
    "src": "9997912345678",
    "dst": "200",
    "calldate": "2025-07-25 15:30:00"
  }
}
```

If `recording_exists` is `false`, the file was deleted or never created.

11. Webhooks

The module fires TWO webhook event types to the campaign's `callbackUrl`. Both events go to the SAME URL — your receiver should switch on the `event` field.

Event 1: `campaign.state_changed`

Fires whenever the campaign's state transitions between `open (0)`, `paused (2)`, or `closed (1)`.

Triggers: - User clicks Resume / Pause / Close on dashboard detail view - API call: `PUT /task/{id}` with `state` field - Worker auto-close (all numbers dialed) - Any other module hook that changes the state

Payload:

```
POST {callbackUrl}
Content-Type: application/json
User-Agent: ModuleAutoDialerManage/1.35 (Bit Dream IT)

{
  "event": "campaign.state_changed",
  "task_id": 42,
  "name": "Payment reminders - June",
  "crm_id": "1001",
  "old_state": 0,
  "new_state": 2,
  "old_state_label": "open",
  "new_state_label": "paused",
  "changed_at": "2025-07-25T15:30:00+03:00"
}
```

State value reference: - `0` = open (active, dialing) - `1` = closed (finished or stopped) - `2` = paused (temporarily on hold)

Event 2: `campaign.completed`

Fires EXACTLY ONCE per close transition (tracked in `$webhookFired` worker memory). If the campaign is re-opened and then closed again, this event will fire again — each close transition produces one event.

Payload:

```
POST {callbackUrl}
Content-Type: application/json
User-Agent: ModuleAutoDialerManage/1.35 (Bit Dream IT)

{
  "event": "campaign.completed",
  "task_id": 42,
  "name": "Payment reminders - June",
  "crm_id": "1001",
  "total": 1500,
  "answered": 872,
  "failed": 628,
  "completed_at": "2025-07-25T15:30:00+03:00"
}
```

Delivery guarantees

- The worker polls the DB approximately once per second
- `state_changed` fires for EVERY transition detected (i.e. whenever the worker sees the state field change)
- `completed` fires EXACTLY ONCE per close. Worker memory tracks which `task_id` s have already received the completion event
- If a campaign is re-opened (state back to 0 or 2), the completion webhook flag is reset — closing it again will fire `completed` again
- HTTP timeout: 5 seconds
- Failed deliveries are logged but NOT retried (no at-least-once guarantee — design your receiver to be tolerant of missed events, e.g. by reconciling state via `GET /task/{id}/status` on a schedule)
- TLS verification is skipped (self-signed certs work)
- Both events are POSTed to the same `callbackUrl` — differentiate via the `event` field

Sample Laravel receiver (handles both events)

```
// routes/api.php
Route::post('/webhooks/dialer', function (\Illuminate\Http\Request $request) {
    $payload = $request->validate([
        'event'      => 'required|string|in:campaign.completed,campaign.state_changed',
        'task_id'    => 'required|integer',
        'name'       => 'required|string',
        'crm_id'     => 'nullable|string',
    ]);

    switch ($payload['event']) {
        case 'campaign.state_changed':
            $data = $request->validate([
                'old_state'      => 'integer',
                'new_state'      => 'integer',
                'old_state_label' => 'string',
                'new_state_label' => 'string',
                'changed_at'     => 'string',
            ]);
            // Update CRM status, notify Slack channel, etc.
            Log::info("Campaign { $payload['task_id'] } state changed", $data);
            Campaign::where('external_id', $payload['task_id'])
                ->update(['status' => $data['new_state_label']]);
            break;

        case 'campaign.completed':
            $data = $request->validate([
                'total'          => 'integer',
                'answered'       => 'integer',
                'failed'         => 'integer',
                'completed_at'   => 'string',
            ]);
            Log::info("Campaign { $payload['task_id'] } completed", $data);
            Campaign::where('external_id', $payload['task_id'])
                ->update([
                    'status'          => 'completed',
                    'total_dialed'    => $data['total'],
                    'total_answered'  => $data['answered'],
                    'completed_at'    => now(),
                ]);
            // Send email summary, generate final report, etc.
            break;
    }

    return response()->json(['ok' => true]);
});
```

Reconciling missed events

If your webhook receiver is down when an event fires, the event is lost (no retry). To detect missed events, poll `GET /task/{id}/status` periodically for active campaigns and compare against your local state.

12. Laravel integration cookbook

Setup

Add to `.env` :

```
MIKOPBX_HOST=http://your-pbx.local  
MIKOPBX_USER=admin  
MIKOPBX_PASS=your_password
```

Add to `config/services.php` :

```
'mikopbx' => [  
    'host' => env('MIKOPBX_HOST', 'http://127.0.0.1'),  
    'user' => env('MIKOPBX_USER', 'admin'),  
    'pass' => env('MIKOPBX_PASS', ''),  
],
```

Service class

```
// app/Services/MikoPBX.php
<?php

namespace App\Services;

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Cache;

class MikoPBX
{
    private string $host;
    private string $user;
    private string $pass;

    public function __construct()
    {
        $this->host = config('services.mikopbx.host');
        $this->user = config('services.mikopbx.user');
        $this->pass = config('services.mikopbx.pass');
    }

    private function client()
    {
        $cookie = $this->getSessionCookie();
        return Http::withHeaders(['Cookie' => "PHPSESSID=$cookie"]);
    }

    private function getSessionCookie(): string
    {
        return Cache::remember('mikopbx_session', now()->addMinutes(30), function () {
            $response = Http::asForm()->post("{ $this->host }/admin-cabinet/session/start", [
                'login' => $this->user,
                'password' => $this->pass,
            ]);
            return $response->cookie('PHPSESSID') ?? '';
        });
    }

    public function createCampaign(array $data): array
    {
        return $this->client()
            ->post("{ $this->host }/pbxcore/api/module-dialer-manage/v1/task", $data)
            ->json();
    }

    public function pauseCampaign(int $taskId): array
    {
        return $this->client()
            ->put("{ $this->host }/pbxcore/api/module-dialer-manage/v1/task/{ $taskId }", [
                'state' => 2,
            ])
            ->json();
    }

    public function resumeCampaign(int $taskId): array
    {
        return $this->client()
            ->put("{ $this->host }/pbxcore/api/module-dialer-manage/v1/task/{ $taskId }", [
                'state' => 0,
            ])
    }
}
```

```

        ->json();
    }

    public function getCampaignStatus(int $taskId): array
    {
        return $this->client()
            ->get("{ $this->host }/pbxcore/api/module-dialer-manage/v1/task/{ $taskId }/
            status")
            ->json();
    }

    public function getSummary(int $taskId): array
    {
        return $this->client()
            ->get("{ $this->host }/pbxcore/api/module-dialer-manage/v1/task/{ $taskId }/
            summary")
            ->json();
    }

    public function addToBlacklist(array $numbers, string $reason = ''): array
    {
        return $this->client()
            ->post("{ $this->host }/pbxcore/api/module-dialer-manage/v1/blacklist", [
                'numbers' => $numbers,
                'reason' => $reason,
                'source' => 'manual',
            ])
            ->json();
    }

    public function syncResults(callable $callback, int $lookbackSeconds = 0): void
    {
        $cacheKey = 'mikopbx_results_cursor';
        $cursor = Cache::get($cacheKey, time() - $lookbackSeconds);

        $response = $this->client()
            ->get("{ $this->host }/pbxcore/api/module-dialer-manage/v1/results/{ $cursor }")
            ->json();

        if ($response['result'] ?? false) {
            foreach ($response['data']['results'] ?? [] as $row) {
                $callback($row);
                if (isset($row['changeTime']) && $row['changeTime'] > $cursor) {
                    $cursor = $row['changeTime'];
                }
            }
            Cache::put($cacheKey, $cursor, now()->addDays(7));
        }
    }
}

```

Usage examples


```
// 1. Launch a campaign
Sapi = app(App\Services\MikoPBX::class);
$result = $sapi->createCampaign([
    'crmId' => 'INV-2025-001',
    'name' => 'Invoice reminders - July',
    'state' => 0,
    'innerNum' => '200',
    'innerNumType' => 'exten',
    'maxCountChannels' => 5,
    'dialPrefix' => '999',
    'timeStart' => 540,
    'timeEnd' => 1080,
    'scheduleDays' => '1,2,3,4,5',
    'amdEnabled' => 1,
    'callbackUrl' => route('webhooks.dialer'),
    'numbers' => $customers->map(fn($c) => [
        'number' => $c->phone,
        'params' => ['speech' => "Hello {$c->name}, your invoice of {$c->balance} is
        due."],
    ]->toArray(),
]);

// 2. Scheduled task: sync results every minute
// app/Console/Commands/SyncDialerResults.php
class SyncDialerResults extends Command
{
    protected $signature = 'dialer:sync';
    public function handle(App\Services\MikoPBX $sapi)
    {
        $sapi->syncResults(function ($row) {
            CallResult::updateOrCreate(
                ['linked_id' => $row['linkedId']],
                [
                    'task_id' => $row['task_id'],
                    'phone' => $row['number'],
                    'state' => $row['outDialState'],
                    'duration' => $row['duration'],
                    'cause' => $row['cause'],
                    'happened_at' => $row['time'],
                ]
            );
        }, lookbackSeconds: 86400);
    }
}

// 3. Webhook receiver – handles both campaign.state_changed and campaign.completed
// routes/api.php
Route::post('/webhooks/dialer', function (\Illuminate\Http\Request $request) {
    $payload = $request->validate([
        'event' => 'required|string|in:campaign.state_changed,campaign.completed',
        'task_id' => 'required|integer',
        'name' => 'required|string',
        'crm_id' => 'nullable|string',
    ]);

    switch ($payload['event']) {
        case 'campaign.state_changed':
            $data = $request->validate([
                'old_state' => 'integer', 'new_state' => 'integer',
                'old_state_label' => 'string', 'new_state_label' => 'string',
            ]);
    }
});
```

```

        'changed_at' => 'string',
    ]);
    Campaign::where('external_id', $payload['task_id'])
        ->update(['status' => $data['new_state_label']]);
    Log::info("Campaign {$payload['task_id']}: {$data['old_state_label']} →
    {$data['new_state_label']}");
    break;

case 'campaign.completed':
    $data = $request->validate([
        'total' => 'integer', 'answered' => 'integer',
        'failed' => 'integer', 'completed_at' => 'string',
    ]);
    Campaign::where('external_id', $payload['task_id'])
        ->update([
            'status' => 'completed',
            'total_dialed' => $data['total'],
            'total_answered' => $data['answered'],
            'completed_at' => now(),
        ]);
    // Trigger final report email, Slack notification, etc.
    break;
}
return response()->json(['ok' => true]);
});

```

13. Plain PHP integration

```

<?php
// dialer-client.php – minimal PHP client, no framework required

class DialerClient {
    private string $baseUrl;
    private ?string $cookieFile = null;

    public function __construct(string $host, string $user = '', string $pass = '') {
        $this->baseUrl = rtrim($host, '/');
        if ($user && $pass) {
            $this->login($user, $pass);
        }
    }

    private function login(string $user, string $pass): void {
        $this->cookieFile = tempnam(sys_get_temp_dir(), 'dialer_cookie_');
        $ch = curl_init("{ $this->baseUrl }/admin-cabinet/session/start");
        curl_setopt_array($ch, [
            CURLOPT_POST => true,
            CURLOPT_POSTFIELDS => http_build_query(['login' => $user, 'password' =>
                $pass]),
            CURLOPT_COOKIEJAR => $this->cookieFile,
            CURLOPT_RETURNTRANSFER => true,
        ]);
        curl_exec($ch);
        curl_close($ch);
    }

    public function request(string $method, string $path, ?array $body = null): array {
        $ch = curl_init("{ $this->baseUrl }{$path}");
        $opts = [
            CURLOPT_CUSTOMREQUEST => $method,
            CURLOPT_RETURNTRANSFER => true,
            CURLOPT_HTTPHEADER => ['Content-Type: application/json'],
        ];
        if ($this->cookieFile) {
            $opts[CURLOPT_COOKIEFILE] = $this->cookieFile;
        }
        if ($body !== null) {
            $opts[CURLOPT_POSTFIELDS] = json_encode($body, JSON_UNESCAPED_UNICODE);
        }
        curl_setopt_array($ch, $opts);
        $response = curl_exec($ch);
        curl_close($ch);
        return json_decode($response, true) ?: [];
    }

    public function createCampaign(array $data): array {
        return $this->request('POST', '/pbxcore/api/module-dialer-manage/v1/task', $data);
    }

    public function pause(int $taskId): array {
        return $this->request('PUT', "/pbxcore/api/module-dialer-manage/v1/task/{ $taskId }", ['state' => 2]);
    }

    public function getStatus(int $taskId): array {
        return $this->request('GET', "/pbxcore/api/module-dialer-manage/v1/task/{ $taskId }/status");
    }
}

```

```

    public function getSummary(int $taskId): array {
        return $this->request('GET', "/pbxcore/api/module-dialer-manage/v1/task/{$_taskId}/summary");
    }

    public function addToBlacklist(array $numbers, string $reason = ''): array {
        return $this->request('POST', '/pbxcore/api/module-dialer-manage/v1/blacklist', [
            'numbers' => $numbers,
            'reason' => $reason,
            'source' => 'manual',
        ]);
    }
}

// Usage (local, no auth needed):
$client = new DialerClient('http://127.0.0.1');
$result = $client->createCampaign([
    'crmId' => 'test-1',
    'name' => 'Test campaign',
    'state' => 0,
    'innerNum' => '200',
    'innerNumType' => 'exten',
    'maxCountChannels' => 1,
    'numbers' => [['number' => '7912345678']],
]);
print_r($result);

```

14. Python integration

```

import requests

class DialerClient:
    def __init__(self, host, user=None, password=None):
        self.base_url = host.rstrip('/')
        self.session = requests.Session()
        if user and password:
            self.session.post(
                f"{self.base_url}/admin-cabinet/session/start",
                data={'login': user, 'password': password}
            )

    def create_campaign(self, data):
        r = self.session.post(
            f"{self.base_url}/pbxcore/api/module-dialer-manage/v1/task",
            json=data
        )
        return r.json()

    def pause(self, task_id):
        r = self.session.put(
            f"{self.base_url}/pbxcore/api/module-dialer-manage/v1/task/{task_id}",
            json={'state': 2}
        )
        return r.json()

    def get_status(self, task_id):
        r = self.session.get(
            f"{self.base_url}/pbxcore/api/module-dialer-manage/v1/task/{task_id}/status"
        )
        return r.json()

    def sync_results(self, callback, lookback_seconds=86400):
        import time
        cursor = int(time.time()) - lookback_seconds
        while True:
            r = self.session.get(
                f"{self.base_url}/pbxcore/api/module-dialer-manage/v1/results/{cursor}"
            ).json()
            if not r.get('result'):
                break
            results = r.get('data', {}).get('results', [])
            if not results:
                break
            for row in results:
                callback(row)
                if row.get('changeTime', 0) > cursor:
                    cursor = row['changeTime']

# Usage
client = DialerClient('http://127.0.0.1')
result = client.create_campaign({
    'crmId': 'py-test-1',
    'name': 'Python test',
    'state': 0,
    'innerNum': '200',
    'innerNumType': 'exten',
    'maxCountChannels': 1,
    'numbers': [{'number': '7912345678'}],

```

```
}
print(result)
```

15. Error handling

Common errors

HTTP code	result	Cause	Fix
200	false	Invalid JSON body	Check request body
200	false	"Task {id} not found"	Check task ID
200	false	"Field 'innerNum' is required"	Add required field
200	false	"No file uploaded"	Add <code>file</code> field to multipart
401	-	Session expired	Re-login
404	-	Wrong URL	Check base URL
500	-	PHP exception	Check <code>/var/log/mikopbx/ModuleAutoDialerManage/*.log</code>

Defensive client pattern

```
$response = $client->createCampaign($data);
if (!$response['result'] ?? false) {
    $errors = $response['messages'] ?? ['Unknown error'];
    throw new \RuntimeException('Dialer API error: ' . implode(', ', $errors));
}
$taskId = $response['data']['id'] ?? null;
```


16. Rate limiting & performance

Recommended polling intervals

Endpoint	Recommended interval	Notes
<code>/task/{id}/status</code>	3-5s	Lightweight DB count
<code>/agents-status</code>	3-5s	Reads from Redis cache
<code>/results/{changeTime}</code>	3-5s	Incremental, only new rows
<code>/task/{id}/summary</code>	30-60s	Heavy aggregate query
<code>/blacklist</code>	on demand	Paginated

Concurrency limits

- Max simultaneous calls per campaign: controlled by `maxCountChannels` field
- No hard limit on number of campaigns
- Worker processes 1 campaign iteration per second (tunable in `WorkerDialer::start()`)

Database performance

- `m_ModuleAutoDialerManage_TaskResults` is the largest table. Index on `(task_id, changeTime)` is auto-created by Phalcon annotations
 - For > 1M rows, consider partitioning by month
-

17. Schema reference

Tasks table

```
CREATE TABLE m_ModuleAutoDialerManage_Tasks (
  id INT PRIMARY KEY AUTO_INCREMENT,
  crmId VARCHAR(255),
  name VARCHAR(255),
  description TEXT,
  innerNum VARCHAR(50),
  innerNumType VARCHAR(20) DEFAULT 'exten',
  maxCountChannels INT NOT NULL DEFAULT 1,
  maxAttempt INT DEFAULT 1,
  attemptUntilSignal INT DEFAULT 0,
  tryInterval INT DEFAULT 60,
  timeStart INT DEFAULT 0,
  timeEnd INT DEFAULT 1440,
  scheduleDays VARCHAR(20) DEFAULT '',
  state VARCHAR(10) DEFAULT '0',
  dialPrefix VARCHAR(20) DEFAULT '',
  isCallback INT DEFAULT 0,
  amdEnabled INT DEFAULT 0,
  callbackUrl TEXT,
  INDEX (crmId),
  INDEX (state),
  INDEX (timeStart),
  INDEX (timeEnd)
);
```

Blacklist table

```
CREATE TABLE m_ModuleAutoDialerManage_Blacklist (
  id INT PRIMARY KEY AUTO_INCREMENT,
  number VARCHAR(50) NOT NULL,
  reason VARCHAR(255) DEFAULT '',
  source VARCHAR(50) DEFAULT 'manual',
  createdAt INT NOT NULL,
  UNIQUE INDEX (number)
);
```

TaskResults table

```
CREATE TABLE m_ModuleAutoDialerManage_TaskResults (
  id INT PRIMARY KEY AUTO_INCREMENT,
  task_id INT NOT NULL,
  number VARCHAR(50),
  outDialState VARCHAR(20),
  inDialState VARCHAR(20),
  duration INT DEFAULT 0,
  attempt INT DEFAULT 1,
  time VARCHAR(30),
  changeTime INT,
  cause VARCHAR(255),
  linkedId VARCHAR(100),
  INDEX (task_id, changeTime),
  INDEX (number)
);
```

PolingResults table

```
CREATE TABLE m_ModuleAutoDialerManage_PolingResults (
  id INT PRIMARY KEY AUTO_INCREMENT,
  pollingId INT NOT NULL,
  questionId INT NOT NULL,
  crmId INT,
  number VARCHAR(50),
  value VARCHAR(255),
  `key` VARCHAR(10),
  time VARCHAR(30),
  changeTime INT,
  INDEX (pollingId, changeTime),
  INDEX (number)
);
```

API endpoint summary

Method	Endpoint	Purpose
GET	/task	List campaigns
POST	/task	Create campaign
GET	/task/{id}	Get campaign
PUT	/task/{id}	Update campaign
DELETE	/task/{id}	Delete campaign
GET	/task/{id}/status	Live status (polling)
GET	/task/{id}/summary	Summary report
GET	/task/{id}/export	CSV export
POST	/task/{id}/import-csv	CSV import
POST	/task/{id}/test-call	Test call single number
POST	/task-signal-close	Batch close
GET	/results/{changeTime}	Call results (incremental)
GET	/polling-results/{changeTime}	Poll results (incremental)
GET	/polling	List surveys
POST	/polling	Create survey
GET	/polling/{id}	Get survey
DELETE	/polling/{id}	Delete survey
POST	/audio	Upload audio file
GET	/audio	List audio files
DELETE	/audio/{name}	Delete audio file
GET	/agents-status	Agent panel (polling)
POST	/blacklist	Add to DNC
GET	/blacklist	List DNC (paginated)
DELETE	/blacklist/{number}	Remove from DNC
GET	/recording/{linkedId}	Find recording file
GET	/client-by-phone/{phone}	Find client by phone
POST	/client	Add client
DELETE	/client/{id}	Delete client

Method	Endpoint	Purpose
POST	/upload-xls	Upload XLS numbers
POST	/crm-test	Test CRM connection
GET	/test	Health check